Vanya Thakur, A2305223337

INDEX

S No.	Title	Date	Signature
1	Write a program to check if a given string is accepted or rejected by production rules S → aS | Sb | ab	05/12/2025	
2	To convert an infix expression into postfix expression using stack	12/01/2025	
3	To count the number of tokens in a given program	19/12/2025	
4	To remove left recursion from a given grammar	09/01/2026	
5	To convert infix expression into prefix expression	16/01/2026	
6	To convert a regular expression into finite automata	23/01/2026	
7	To convert a given NFA into DFA	23/01/2026	
8	To find FIRST and FOLLOW of a given CFG	30/01/2026	
9	Write a program to implement a shift reduce parser for the given grammar	30/01/2026	
10	Write a program which accepts a regular grammar with no left recursion and no null production rules and checks whether a given string is accepted	13/02/2026	
11	Write separate programs for each regular expression mentioned	13/02/2026	
12	Write a program to implement a recursive descent calculator	20/02/2026	
13	Design a parser which accepts a string and tells whether the string is accepted by given grammar or not	20/02/2026	
14	Design a parser which accepts a mathematical expression (containing integers only). If the expression is valid, then evaluate the expression else report that the expression is invalid.	06/03/2026	
15	Designing of various types of parsers.	06/03/2026	

Vanya Thakur, A2305223337

EXPERIMENT 1

AIM: To write a C program to check whether a given string is accepted or rejected by the production rules S → aS | Sb | ab

SOFTWARE USED: VS Code

THEORY:

A grammar consists of a set of production rules used to generate strings of a language. The given grammar S → aS | Sb | ab generates strings that start with one or more a symbols and end with one or more b symbols. The production ab acts as the base condition for valid strings. Recursive productions allow repetition of characters on either side. To verify whether a string belongs to this grammar, logical checks are used instead of actual derivation. The program validates the structure of the string based on the grammar rules. If the string follows the required pattern, it is accepted; otherwise, it is rejected.

ALGORITHM:

BEGIN

INPUT string

SET i ← 0

SET a_count ← 0

SET b_count ← 0

SET n ← length of string

IF n = 0 THEN

  PRINT "String is rejected"

  STOP

END IF

WHILE i < n AND string[i] = 'a' DO

  a_count ← a_count + 1

  i ← i + 1

END WHILE

WHILE i < n AND string[i] = 'b' DO

  b_count ← b_count + 1

  i ← i + 1

END WHILE

IF i = n AND a_count > 0 AND b_count > 0 THEN

  PRINT "String is accepted"

ELSE

  PRINT "String is rejected"

END IF

END

CODE:

```
#include <stdio.h>

#include <string.h>


void check(char str[]) {

    int i = 0, n = strlen(str);

    int a_count = 0;

    int b_count = 0;


    if (n == 0) {
```

```
        printf("String is rejected.\n");
        return;
    }

    while (i < n && str[i] == 'a') {
        i++;
        a_count++;
    }

    while (i < n && str[i] == 'b') {
        i++;
        b_count++;
    }

    if (i == n && a_count > 0 && b_count > 0) {
        printf("String is accepted.\n");
    } else {
        printf("String is rejected.\n");
    }
}

int main() {
    char str[100];
    printf("Enter a string: ");
    scanf("%s", str);
```

```
    check(str);
    return 0;
}
```

OUTPUT:

```
Enter a string: abcghh
String is rejected.
```

CONCLUSION:

The program correctly determines whether a given string is accepted or rejected by verifying if it follows the grammar rule S → aS | Sb | ab.

Vanya Thakur, A2305223337

EXPERIMENT 2

AIM: To convert an infix expression into postfix expression using stack.

SOFTWARE USED: VS Code

THEORY:

Infix expressions are easy for humans to read but difficult for machines to evaluate due to operator precedence. Postfix expressions eliminate ambiguity by placing operators after operands. A stack is used to temporarily store operators while scanning the infix expression. Operator precedence determines when operators are pushed or popped from the stack. Operands are directly added to the postfix expression. This method ensures correct order of evaluation. Stack-based conversion is efficient and widely used in compilers.

ALGORITHM:

BEGIN

INPUT infix_expression

INITIALIZE empty stack

SET top ← -1

SET i ← 0

WHILE infix_expression[i] is not end of string DO

  IF infix_expression[i] is an operand THEN

    PRINT infix_expression[i]

  ELSE

    WHILE stack is not empty AND

      precedence(top element of stack) ≥ precedence(infix_expression[i]) DO

```
            PRINT pop element from stack
        END WHILE
        PUSH infix_expression[i] onto stack
    END IF
    i ← i + 1
END WHILE
WHILE stack is not empty DO
    PRINT pop element from stack
END WHILE
END
```

CODE:

```
#include <stdio.h>
#include <ctype.h>
char stack[50];
int top = -1;
void push(char x) {
    stack[++top] = x;
}
char pop() {
    return stack[top--];
}
int precedence(char x) {
    if (x == '+' || x == '-') return 1;
    if (x == '*' || x == '/') return 2;
```

```
    return 0;
}
int main() {
    char exp[50], *e;
    printf("Enter infix expression: ");
    scanf("%s", exp);
    e = exp;
    while (*e) {
        if (isalnum(*e))
            printf("%c", *e);
        else {
            while (top != -1 && precedence(stack[top]) >= precedence(*e))
                printf("%c", pop());
            push(*e);
        }
        e++;
    }
    while (top != -1)
        printf("%c", pop());
    return 0;
}
```

OUTPUT:

```
Infix expression: a+b*(c^d-e)^(f+g*h)-i
Postfix expression: abcd^e-fgh*+^*+i-
```

CONCLUSION:

A Infix expression is converted into postfix expression successfully using stack

Vanya Thakur, A2305223337

EXPERIMENT 3

AIM: To count the number of tokens in a given program.

SOFTWARE USED: VS Code

THEORY:

Tokens are the smallest meaningful units of a programming language. They include keywords, identifiers, operators, constants, and symbols. Tokenization is the first phase of compiler design known as lexical analysis. A program is divided into tokens based on spaces and delimiters. Counting tokens helps analyze program structure and complexity. It is useful in syntax analysis and compiler optimization. This program counts tokens present in a given input.

ALGORITHM:

BEGIN

INPUT a line of code

SET token_count ← 0

CREATE a stream from the input line

WHILE there is a token available in the stream DO

  READ the next token

  token_count ← token_count + 1

END WHILE

PRINT "Number of tokens: ", token_count

END

CODE:

```
#include <iostream>

#include <sstream>

using namespace std;

int main() {

    string line, token;

    int count = 0;

    cout << "Enter a string of code:\n";

    getline(cin, line);

    stringstream ss(line);

    while (ss >> token)

        count++;

    cout << "Number of tokens: " << count << endl;

    return 0;

}
```

OUTPUT:

```
Enter a string of code: compiler class sem 6
Number of tokens: 4
```

CONCLUSION: The number of tokens are counted in the program.

Vanya Thakur, A2305223337

EXPERIMENT 4

AIM: To remove left recursion from a given grammar.

SOFTWARE USED: VS Code

THEORY:

Left recursion occurs when a non-terminal appears on the leftmost side of its own production. This causes infinite recursion in top-down parsers. To avoid this issue, left recursion must be eliminated. The grammar is rewritten using a new non-terminal symbol. The transformation preserves the language generated by the grammar. Removing left recursion improves parsing efficiency. This process is essential in compiler design.

ALGORITHM:

BEGIN

INPUT non_terminal A

INPUT alpha

INPUT beta

PRINT A → beta A'

PRINT A' → alpha A' | ε

END

CODE:

```
#include <iostream>

using namespace std;
```

```
int main() {
    char A, alpha[10], beta[10];
    cout << "Enter Non-Terminal: ";
    cin >> A;
    cout << "Enter alpha: ";
    cin >> alpha;
    cout << "Enter beta: ";
    cin >> beta;
    cout << A << " -> " << beta << A << "'\n";
    cout << A << "' -> " << alpha << A << "' | ε\n";
    return 0;
}
```

OUTPUT:

```
Enter the production (e.g., A->Aa|b): A->Aa|b

Grammar after removing left recursion:
A -> bA'
A' -> aA'|e
```

CONCLUSION: Left recursion is successfully removed from the grammar.

Vanya Thakur, A2305223337

EXPERIMENT 5

AIM: To convert infix expression into prefix expression.

SOFTWARE USED: VS Code

THEORY:

Prefix notation places operators before operands and eliminates the need for parentheses. It simplifies expression evaluation for machines. Conversion from infix to prefix is performed using stack techniques. The infix expression is reversed and converted to postfix. The resulting postfix expression is reversed to obtain prefix form. Operator precedence is maintained throughout the process. This method ensures correct evaluation order.

ALGORITHM:

BEGIN

INPUT infix_expression

REVERSE the infix_expression

FOR each character in infix_expression DO

  IF character = '(' THEN

    replace it with ')'

  ELSE IF character = ')' THEN

    replace it with '('

  END IF

END FOR

INITIALIZE empty stack

INITIALIZE postfix_expression ← empty

```
FOR each character c in infix_expression DO
   IF c is an operand THEN
      append c to postfix_expression
   ELSE IF c = '(' THEN
      PUSH c onto stack
   ELSE IF c = ')' THEN
      WHILE top of stack ≠ '(' DO
         append pop(stack) to postfix_expression
      END WHILE
      POP '(' from stack
   ELSE
      WHILE stack not empty AND precedence(top of stack) ≥ precedence(c) DO
         append pop(stack) to postfix_expression
      END WHILE
      PUSH c onto stack
   END IF
END FOR
WHILE stack not empty DO
   append pop(stack) to postfix_expression
END WHILE
REVERSE postfix_expression
PRINT postfix_expression as prefix_expression
END
```

CODE:

```
#include <iostream>

#include <string>

#include <stack>

#include <algorithm>

#include <cctype>


int getPrecedence(char c) {

    if (c == '^')

        return 3;

    if (c == '*' || c == '/')

        return 2;

    if (c == '+' || c == '-')

        return 1;

    return -1;

}


std::string infixToPrefix(std::string infix) {

    std::reverse(infix.begin(), infix.end());


    for (size_t i = 0; i < infix.length(); ++i) {

        if (infix[i] == '(') {

            infix[i] = ')';

        } else if (infix[i] == ')') {

            infix[i] = '(';
```

```
    }
}

std::stack<char> s;
std::string postfix;

for (size_t i = 0; i < infix.length(); ++i) {
    char c = infix[i];

    if (isalnum(c)) {
        postfix += c;
    } else if (c == '(') {
        s.push(c);
    } else if (c == ')') {
        while (!s.empty() && s.top() != '(') {
            postfix += s.top();
            s.pop();
        }
        if (!s.empty()) {
            s.pop();
        }
    } else {
        if (c == '^') {
            while (!s.empty() && s.top() != '(' && getPrecedence(s.top()) >= getPrecedence(c)) {
                postfix += s.top();
```

```
                s.pop();
            }
        } else {
            while (!s.empty() && s.top() != '(' && getPrecedence(s.top()) > getPrecedence(c)) {
                postfix += s.top();
                s.pop();
            }
        }
        s.push(c);
    }
  }

  while (!s.empty()) {
    postfix += s.top();
    s.pop();
  }

  std::reverse(postfix.begin(), postfix.end());
  return postfix;
}

int main() {
  std::string infix_expression;
  std::cout << "Enter an infix expression: ";
  std::getline(std::cin, infix_expression);
```

```
    std::cout << "Prefix expression: " << infixToPrefix(infix_expression) << std::endl;

    return 0;
}
```

OUTPUT:

```
Enter an infix expression: (A+B|C+D*(F-H)^A*B)
Prefix expression: |+AB+C**D^-FHAB
```

CONCLUSION:

The program converts an infix expression into prefix notation.

Vanya Thakur, A2305223337

EXPERIMENT 6

AIM: To convert a regular expression into finite automata.

SOFTWARE USED: VS Code

THEORY:

A regular expression describes patterns used to define languages. Finite automata are abstract machines that recognize these patterns. Every regular expression can be converted into an equivalent finite automaton.

NFAs are preferred because they allow ε-transitions and multiple transitions. Each symbol and operator in the regular expression contributes to state transitions. The constructed automaton accepts the same language. This concept is widely used in lexical analysis.

ALGORITHM:

BEGIN

INPUT regular_expression

INSERT explicit concatenation operator '.' wherever required in the expression

CONVERT the modified infix expression into postfix expression using stack

INITIALIZE empty stack for NFA fragments

INITIALIZE state_count ← 0

FOR each symbol c in postfix expression DO

  IF c is an operand THEN

    CREATE two new states s and e

    ADD transition s → e with symbol c

    PUSH NFA(s, e) onto stack

ELSE IF c = '.' THEN

POP NFA b

POP NFA a

ADD ε-transition from a.end → b.start

CREATE new NFA with start = a.start and end = b.end

PUSH this NFA onto stack

ELSE IF c = '|' THEN

POP NFA b

POP NFA a

CREATE new states s and e

ADD ε-transition s → a.start

ADD ε-transition s → b.start

ADD ε-transition a.end → e

ADD ε-transition b.end → e

PUSH NFA(s, e) onto stack

ELSE IF c = '*' THEN

POP NFA a

CREATE new states s and e

ADD ε-transition s → a.start

ADD ε-transition s → e

ADD ε-transition a.end → a.start

ADD ε-transition a.end → e

PUSH NFA(s, e) onto stack

END FOR

POP the final NFA from stack

PRINT start state

PRINT final state

PRINT all transitions of the automaton

END

CODE:

```
#include <iostream>

#include <stack>

#include <vector>

#include <string>

#include <cctype>

using namespace std;


struct Transition{

    int from;

    int to;

    char symbol;

};


struct NFA{

    int start;

    int end;

};


vector<Transition> transitions;
```

```
int stateCount=0;


int prec(char c){
    if(c=='*') return 3;
    if(c=='.') return 2;
    if(c=='|') return 1;
    return 0;
}


string addConcat(string re){
    string res="";
    for(int i=0;i<re.size();i++){
        res+=re[i];
        if(i+1<re.size()){
            char a=re[i];
            char b=re[i+1];
            if((isalnum(a)||a=='*'||a==')')&&(isalnum(b)||b=='('))
                res+='.';
        }
    }
    return res;
}


string infixToPostfix(string re){
    stack<char> st;
```

```
    string post="";
    for(char c:re){
        if(isalnum(c)) post+=c;
        else if(c=='(') st.push(c);
        else if(c==')'){
            while(!st.empty()&&st.top()!='('){
                post+=st.top();
                st.pop();
            }
            if(!st.empty()) st.pop();
        }
        else{
            while(!st.empty()&&prec(st.top())>=prec(c)){
                post+=st.top();
                st.pop();
            }
            st.push(c);
        }
    }
    while(!st.empty()){
        post+=st.top();
        st.pop();
    }
    return post;
}
```

```
NFA symbolNFA(char c){
    int s=stateCount++;
    int e=stateCount++;
    transitions.push_back({s,e,c});
    return {s,e};
}

NFA concatNFA(NFA a,NFA b){
    transitions.push_back({a.end,b.start,'e'});
    return {a.start,b.end};
}

NFA unionNFA(NFA a,NFA b){
    int s=stateCount++;
    int e=stateCount++;
    transitions.push_back({s,a.start,'e'});
    transitions.push_back({s,b.start,'e'});
    transitions.push_back({a.end,e,'e'});
    transitions.push_back({b.end,e,'e'});
    return {s,e};
}

NFA starNFA(NFA a){
    int s=stateCount++;
```

```
    int e=stateCount++;
    transitions.push_back({s,a.start,'e'});
    transitions.push_back({s,e,'e'});
    transitions.push_back({a.end,a.start,'e'});
    transitions.push_back({a.end,e,'e'});
    return {s,e};
}

int main(){
    string re;
    cout<<"Enter Regular Expression: ";
    cin>>re;

    re=addConcat(re);
    string postfix=infixToPostfix(re);

    stack<NFA> st;

    for(char c:postfix){
        if(c=='*'){
            NFA a=st.top();
            st.pop();
            st.push(starNFA(a));
        }
        else if(c=='.'){
```

```
            NFA b=st.top(); st.pop();
            NFA a=st.top(); st.pop();
            st.push(concatNFA(a,b));
        }
        else if(c=='|'){
            NFA b=st.top(); st.pop();
            NFA a=st.top(); st.pop();
            st.push(unionNFA(a,b));
        }
        else{
            st.push(symbolNFA(c));
        }
    }

    NFA result=st.top();

    cout<<"\nStart State: "<<result.start<<endl;
    cout<<"Final State: "<<result.end<<endl;
    cout<<"\nTransitions:\n";

    for(auto t:transitions){
        cout<<t.from<<" -- "<<t.symbol<<" --> "<<t.to<<endl;
    }
    return 0;
}
```

OUTPUT:

```
Enter Regular Expression: abc*

Start State: 0
Final State: 7

Transitions:
0 -- a --> 1
2 -- b --> 3
1 -- e --> 2
4 -- c --> 5
6 -- e --> 4
6 -- e --> 7
5 -- e --> 4
5 -- e --> 7
3 -- e --> 6
```

CONCLUSION:

The regular expression is successfully converted into finite automata.

Vanya Thakur, A2305223337

EXPERIMENT 7

AIM: To convert a given NFA into DFA.

SOFTWARE USED: VS Code

THEORY:

An NFA allows multiple transitions for the same input symbol, whereas a DFA allows only one. Although NFAs are easy to design, DFAs are simpler to implement. Therefore, NFAs are converted into equivalent DFAs. The subset construction method is used for conversion. Each DFA state represents a set of NFA states. The resulting DFA accepts the same language. This conversion is fundamental in automata theory.

ALGORITHM:

BEGIN

INPUT number_of_states of NFA

INPUT number_of_input_symbols

INPUT the input symbols

INPUT number_of_transitions

FOR each transition DO

  INPUT from_state, symbol, to_state

  STORE the transition in NFA transition table

END FOR

INPUT start_state

INPUT number_of_final_states

INPUT the final states of NFA

```
CREATE start_set containing start_state
ADD start_set to list_of_DFA_states
FOR each DFA_state in list_of_DFA_states DO
  FOR each input_symbol DO
    CREATE empty set next_state
    FOR each state in current DFA_state DO
      ADD all NFA transitions of (state, input_symbol) to next_state
    END FOR
    STORE DFA transition from current_state using input_symbol to next_state
    IF next_state is not empty AND not already in list_of_DFA_states THEN
      ADD next_state to list_of_DFA_states
    END IF
  END FOR
END FOR
PRINT DFA transition table
FOR each DFA_state DO
  IF it contains any NFA final state THEN
    MARK it as DFA final state
  END IF
END FOR
PRINT DFA final states
END
```

CODE:

```
#include <iostream>

#include <set>

#include <map>

#include <vector>

#include <algorithm>

using namespace std;


int main(){

    int n,m,t;

    cout<<"Enter number of NFA states: ";

    cin>>n;


    cout<<"Enter number of input symbols: ";

    cin>>m;


    vector<char> symbols(m);

    cout<<"Enter input symbols:\n";

    for(int i=0;i<m;i++) cin>>symbols[i];


    map<pair<int,char>, set<int>> nfa;

    cout<<"Enter number of transitions: ";

    cin>>t;


    cout<<"Enter transitions (from symbol to):\n";
```

```
    for(int i=0;i<t;i++){
        int from,to;
        char sym;
        cin>>from>>sym>>to;
        nfa[{from,sym}].insert(to);
    }

    int start;
    cout<<"Enter start state: ";
    cin>>start;

    int f;
    cout<<"Enter number of final states: ";
    cin>>f;

    set<int> finalStates;
    cout<<"Enter final states:\n";
    for(int i=0;i<f;i++){
        int x;
        cin>>x;
        finalStates.insert(x);
    }

    map<set<int>, map<char,set<int>>> dfa;
    vector<set<int>> states;
```

```
    set<int> startSet={start};
    states.push_back(startSet);

    for(int i=0;i<states.size();i++){
        set<int> current=states[i];
        for(char sym:symbols){
            set<int> next;
            for(int s:current){
                next.insert(nfa[{s,sym}].begin(), nfa[{s,sym}].end());
            }
            dfa[current][sym]=next;
            if(next.size() && find(states.begin(),states.end(),next)==states.end()){
                states.push_back(next);
            }
        }
    }

    cout<<"\nDFA Transition Table:\n";
    for(auto st:states){
        cout<<"{ ";
        for(int x:st) cout<<x<<" ";
        cout<<"}";
        for(char sym:symbols){
            cout<<" --"<<sym<<"--> { ";
            for(int x:dfa[st][sym]) cout<<x<<" ";
```

```
            cout<<"}";
        }
        cout<<endl;
    }

    cout<<"\nDFA Final States:\n";
    for(auto st:states){
        for(int x:st){
            if(finalStates.count(x)){
                cout<<"{ ";
                for(int y:st) cout<<y<<" ";
                cout<<"}\n";
                break;
            }
        }
    }

    return 0;
}
```

OUTPUT:

```
Enter number of NFA states: 3
Enter number of input symbols: 2
Enter input symbols:
a b
Enter number of transitions: 4
Enter transitions (from symbol to):
0 a 0
0 a 1
0 b 0
1 b 2
Enter start state: 0
Enter number of final states: 1
Enter final states:
2

DFA Transition Table:
{ 0 } --a--> { 0 1 } --b--> { 0 }
{ 0 1 } --a--> { 0 1 } --b--> { 0 2 }
{ 0 2 } --a--> { 0 1 } --b--> { 0 }

DFA Final States:
{ 0 2 }
```

CONCLUSION: The NFA is successfully converted into an equivalent DFA.

Vanya Thakur, A2305223337

EXPERIMENT 8

AIM: To find FIRST and FOLLOW of a given CFG.

SOFTWARE USED: VS Code

THEORY:

In Compiler Design, FIRST and FOLLOW sets are used in syntax analysis to construct predictive parsing tables.

The FIRST(α) of a grammar symbol α is the set of terminals that appear at the beginning of strings derived from α.

Example
If
$A \rightarrow aB \mid b$

Then FIRST(A) = {a, b}

FOLLOW(A) is the set of terminals that appear **immediately to the right of A** in some sentential form.

Example

$S \rightarrow AB$

FOLLOW(A) = FIRST(B)

If A is at end:

FOLLOW(A) = FOLLOW(S)

These sets help build **LL(1) parsers.**

ALGORITHM:

```
BEGIN
INPUT number_of_productions
FOR each production
   INPUT production_string
   EXTRACT left_hand_side = first character
   EXTRACT right_hand_side = substring after '='
   STORE production
END FOR
FOR each non_terminal
   COMPUTE FIRST(non_terminal)
END FOR


FUNCTION FIRST(X)
   IF X is terminal
      RETURN {X}
   FOR each production X → Y1 Y2 ... Yn
      FOR each symbol Yi in RHS
         ADD FIRST(Yi) except ε to FIRST(X)
         IF ε not in FIRST(Yi)
            BREAK
      IF ε exists in all Yi
         ADD ε to FIRST(X)
   END FOR
END FUNCTION
```

```
FOR each non_terminal
   COMPUTE FOLLOW(non_terminal)
END FOR


FUNCTION FOLLOW(A)
   IF A is start symbol
      ADD '$' to FOLLOW(A)
   FOR each production B → αAβ
      ADD FIRST(β) except ε to FOLLOW(A)
      IF β derives ε OR A is at end
         ADD FOLLOW(B) to FOLLOW(A)
   END FOR
END FUNCTION


PRINT FIRST sets
PRINT FOLLOW sets
END
```

CODE:

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include <string>
using namespace std;
```

```
map<char, vector<string>> prod;
map<char, set<char>> firstSet, followSet;
vector<char> nonTerminals;

set<char> findFirst(char c){
    if(!(c>='A' && c<='Z')) return {c};
    if(!firstSet[c].empty()) return firstSet[c];

    for(string s: prod[c]){
        if(s=="#") firstSet[c].insert('#');
        else{
            for(int i=0;i<s.size();i++){
                set<char> temp = findFirst(s[i]);
                for(char x:temp){
                    if(x!='#') firstSet[c].insert(x);
                }
                if(temp.find('#')==temp.end()) break;
                if(i==s.size()-1) firstSet[c].insert('#');
            }
        }
    }
    return firstSet[c];
}
```

```
void findFollow(char c){
    if(c==nonTerminals[0]) followSet[c].insert('$');


    for(auto p:prod){
        char lhs=p.first;
        for(string s:p.second){
            for(int i=0;i<s.size();i++){
                if(s[i]==c){
                    if(i+1<s.size()){
                        set<char> temp=findFirst(s[i+1]);
                        for(char x:temp){
                            if(x!='#') followSet[c].insert(x);
                        }
                        if(temp.find('#')!=temp.end() && c!=lhs){
                            findFollow(lhs);
                            followSet[c].insert(followSet[lhs].begin(),followSet[lhs].end());
                        }
                    }
                    else if(c!=lhs){
                        findFollow(lhs);
                        followSet[c].insert(followSet[lhs].begin(),followSet[lhs].end());
                    }
                }
            }
        }
```

```
    }
}

int main(){
    int n;
    cout<<"Enter number of productions: ";
    cin>>n;

    cout<<"Enter productions (Example: E=TR):\n";

    for(int i=0;i<n;i++){
        string input;
        cin>>input;

        char lhs=input[0];
        string rhs=input.substr(2);

        prod[lhs].push_back(rhs);
        nonTerminals.push_back(lhs);
    }

    for(char nt:nonTerminals)
        findFirst(nt);

    for(char nt:nonTerminals)
```

```
        findFollow(nt);

    cout<<"\nFIRST Sets\n";

    for(char nt:nonTerminals){
        cout<<"FIRST("<<nt<<") = { ";
        for(char x:firstSet[nt]) cout<<x<<" ";
        cout<<"}\n";
    }

    cout<<"\nFOLLOW Sets\n";

    for(char nt:nonTerminals){
        cout<<"FOLLOW("<<nt<<") = { ";
        for(char x:followSet[nt]) cout<<x<<" ";
        cout<<"}\n";
    }

    return 0;
}
```

OUTPUT:

```
Enter number of productions: 3
Enter productions (Example: E=TR):
E=TR
T=+TI
R=TA

FIRST Sets
FIRST(E) = { + }
FIRST(T) = { + }
FIRST(R) = { + }

FOLLOW Sets
FOLLOW(E) = { $ }
FOLLOW(T) = { + }
FOLLOW(R) = { $ }
```

CONCLUSION: The FIRST and FOLLOW set of given CFG is successfully computed using C++ program.

Vanya Thakur, A2305223337

EXPERIMENT 9

AIM: Write a program to implement a shift reduce parser for the grammar
E → E + E
E → E * E
E → id

SOFTWARE USED: VS Code

THEORY:

Shift reduce parsing is a bottom-up parsing technique used in compiler design. The parser reads the input from left to right and constructs the rightmost derivation in reverse. It uses a stack and performs two main operations: shift and reduce. Shift pushes the next input symbol onto the stack. Reduce replaces symbols on the stack that match the right-hand side of a production rule with the left-hand side nonterminal.

ALGORITHM:

BEGIN

INPUT string

INITIALIZE stack as empty

SET pointer i = 0

PRINT "Stack Input Action"

WHILE input not completely processed DO

  SHIFT operation

    push next input symbol to stack

    move pointer to next symbol

    print action as SHIFT

```
    IF stack contains "id"
        replace "id" with "E"
        print action REDUCE E → id
    IF stack contains "E+E"
        replace "E+E" with "E"
        print action REDUCE E → E+E
    IF stack contains "E*E"
        replace "E*E" with "E"
        print action REDUCE E → E*E
END WHILE
IF stack contains only "E"
    PRINT "String Accepted"
ELSE
    PRINT "String Rejected"
END
```

CODE:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string input, stack = "";
    int i = 0;
```

```
    cout << "Enter input string: ";
    cin >> input;

    cout << "\nStack\tInput\tAction\n";

    while (true) {

        if (i < input.length()) {
            stack += input[i];
            input[i] = ' ';
            cout << stack << "\t" << input.substr(i + 1) << "\tShift\n";
            i++;
        }

        if (stack.find("id") != string::npos) {
            int pos = stack.find("id");
            stack.replace(pos, 2, "E");
            cout << stack << "\t" << input.substr(i) << "\tReduce E->id\n";
        }

        if (stack.find("E+E") != string::npos) {
            int pos = stack.find("E+E");
            stack.replace(pos, 3, "E");
            cout << stack << "\t" << input.substr(i) << "\tReduce E->E+E\n";
        }
```

```
        if (stack.find("E*E") != string::npos) {
            int pos = stack.find("E*E");
            stack.replace(pos, 3, "E");
            cout << stack << "\t" << input.substr(i) << "\tReduce E->E*E\n";
        }

        if (i >= input.length())
            break;
    }

    if (stack == "E")
        cout << "\nString Accepted\n";
    else
        cout << "\nString Rejected\n";

    return 0;
}
```

OUTPUT:

```
Enter input string: id+id*id

Stack   Input   Action
i       d+id*id Shift
id      +id*id  Shift
E       +id*id  Reduce E->id
E+      id*id   Shift
E+i     d*id    Shift
E+id    *id     Shift
E+E     *id     Reduce E->id
E       *id     Reduce E->E+E
E*      id      Shift
E*i     d       Shift
E*id            Shift
E*E             Reduce E->id
E               Reduce E->E*E

String Accepted
```

CONCLUSION: The shift reduce parsing technique was successfully implemented.

Vanya Thakur, A2305223337

EXPERIMENT 10

AIM: Write a program which accepts a regular grammar with no left recursion and no null production rules and checks whether a given string is accepted.

SOFTWARE USED: VS Code

THEORY:

A regular grammar is a grammar that generates regular languages. Productions are usually of the form A → aB or A → a. Regular grammars correspond to finite automata and are used to describe simple language structures.

ALGORITHM:

BEGIN

INPUT number_of_productions

FOR each production

  INPUT production_string

  EXTRACT left_hand_side as first character

  EXTRACT right_hand_side after '='

  STORE production

  IF first production

    SET start_symbol

END FOR

INPUT input_string

SET current_states = {start_symbol}

```
FOR each character in input_string
   CREATE empty next_states
   FOR each non_terminal in urrent_states
      FOR each production of that non_terminal
         IF first symbol of RHS matches input character
            IF RHS has two symbols (aB)
               ADD B to next_states
            ELSE IF RHS has one symbol (a)
               ADD final marker to next_states
         END IF
      END FOR
   END FOR
   SET current_states = next_states
END FOR
IF any state in current_states is final
   PRINT "String Accepted"
ELSE
   PRINT "String Rejected"
END
```

CODE:

```
#include <iostream>
#include <map>
#include <vector>
#include <string>
```

```
using namespace std;

int main() {
    int n;
    map<char, vector<string>> prod;
    char start;

    cout<<"Enter number of productions: ";
    cin>>n;

    cout<<"Enter productions (Example: S=aA or S=a):\n";

    for(int i=0;i<n;i++){
        string p;
        cin>>p;
        char lhs=p[0];
        string rhs=p.substr(2);
        prod[lhs].push_back(rhs);
        if(i==0) start=lhs;
    }

    string str;
    cout<<"Enter string: ";
    cin>>str;
```

```
    vector<char> current;
    current.push_back(start);

    for(int i=0;i<str.length();i++){
        vector<char> next;
        for(char nt:current){
            for(string r:prod[nt]){
                if(r[0]==str[i]){
                    if(r.length()==2)
                        next.push_back(r[1]);
                    else
                        next.push_back('#');
                }
            }
        }
        current=next;
    }

    bool accept=false;

    for(char c:current){
        if(c=='#')
            accept=true;
    }
```

```
    if(accept)
        cout<<"String Accepted\n";
    else
        cout<<"String Rejected\n";

    return 0;
}
```

OUTPUT:

```
Enter number of productions: 3
Enter productions (Example: S=aA or S=a):
S=aA
A=bB
B=c
Enter string: abc
String Accepted
```

CONCLUSION: The program successfully checks whether the string belongs to the regular grammar.

Vanya Thakur, A2305223337

EXPERIMENT 11

AIM: Consider the following regular expressions:

1. (0 + 1) * + 0 * 1 *
2. (a b * c + (d e f) + + a * d + e) +
3. ((a + b) * (c + d) *) + + a b * c * d

Write separate programs for each of the regular expressions mentioned above.

SOFTWARE USED: VS Code

THEORY:

A regular expression is used to describe patterns in strings. It is commonly used in lexical analysis to define tokens. Operators used in regular expressions include union (+), closure (*), and concatenation.

ALGORITHM:

BEGIN

INPUT string

SELECT the required regular expression pattern

CREATE a regular expression object using the selected pattern

COMPARE the input string with the regular expression

IF the string matches the regular expression

  PRINT "Accepted"

ELSE

  PRINT "Rejected"

END

Vanya Thakur, A2305223337

CODE (1):

```
#include <iostream>
#include <regex>
using namespace std;

int main() {
    string s;
    cout<<"Enter string: ";
    cin>>s;

    regex r("(0|1)*|0*1*");

    if(regex_match(s,r))
        cout<<"Accepted\n";
    else
        cout<<"Rejected\n";

    return 0;
}
```

OUTPUT (1):

```
Enter string: 01111
Accepted
```

CODE (2):

```
#include <iostream>

#include <regex>

using namespace std;


int main() {

    string s;

    cout<<"Enter string: ";

    cin>>s;


    regex r("(ab*c|(def)+|a*d+e)+");


    if(regex_match(s,r))

        cout<<"Accepted\n";

    else

        cout<<"Rejected\n";


    return 0;

}
```

OUTPUT (2):

```
Enter string: abbbcc
Rejected
```

CODE (3):

```
#include <iostream>

#include <regex>

using namespace std;


int main() {

    string s;

    cout<<"Enter string: ";

    cin>>s;


    regex r("((a|b)*(c|d)*)+|ab*c*d");


    if(regex_match(s,r))

        cout<<"Accepted\n";

    else

        cout<<"Rejected\n";


    return 0;

}
```

OUTPUT (3):

```
Enter string: abcd
Accepted
```

CONCLUSION: The program successfully verifies strings using regular expressions.

Vanya Thakur, A2305223337

EXPERIMENT 12

AIM: Write a program to implement a recursive descent calculator

SOFTWARE USED: VS Code

THEORY:

A recursive descent parser is a top-down parsing technique where each non-terminal in the grammar is implemented as a function. The parser processes the input string from left to right and builds the structure of the expression using recursive procedures.

For arithmetic expressions the grammar can be written as

$E \rightarrow T\ E'$
$E' \rightarrow +\ T\ E' \mid -\ T\ E' \mid \varepsilon$
$T \rightarrow F\ T'$
$T' \rightarrow *\ F\ T' \mid /\ F\ T' \mid \varepsilon$
$F \rightarrow (E) \mid \text{number}$

Here
E represents expression
T represents term
F represents factor

The recursive descent calculator parses the expression and evaluates the result using recursive functions.

ALGORITHM:

BEGIN

INPUT arithmetic_expression

SET position ← 0

```
FUNCTION E
    value ← T
    WHILE next symbol is '+' OR '-'
        operator ← next symbol
        MOVE position
        value2 ← T
        IF operator is '+'
            value ← value + value2
        ELSE
            value ← value - value2
    RETURN value
END FUNCTION


FUNCTION T
    value ← F
    WHILE next symbol is '*' OR '/'
        operator ← next symbol
        MOVE position
        value2 ← F
        IF operator is '*'
            value ← value * value2
        ELSE
            value ← value / value2
    RETURN value
END FUNCTION
```

FUNCTION F

  IF next symbol is '('

    MOVE position

    value ← E

    MOVE position

    RETURN value

  ELSE

    value ← number read from input

    RETURN value

END FUNCTION

CALL E to evaluate expression

PRINT result

END

CODE:

```
#include <iostream>

#include <string>

using namespace std;


string input;

int pos=0;


int E();

int T();
```

```
int F();


int E(){
    int val=T();
    while(pos<input.length() && (input[pos]=='+' || input[pos]=='-')){
        char op=input[pos++];
        int val2=T();
        if(op=='+') val+=val2;
        else val-=val2;
    }
    return val;
}


int T(){
    int val=F();
    while(pos<input.length() && (input[pos]=='*' || input[pos]=='/')){
        char op=input[pos++];
        int val2=F();
        if(op=='*') val*=val2;
        else val/=val2;
    }
    return val;
}


int F(){
```

```
    if(input[pos]=='('){
        pos++;
        int val=E();
        pos++;
        return val;
    }
    int val=0;
    while(pos<input.length() && isdigit(input[pos])){
        val=val*10+(input[pos]-'0');
        pos++;
    }
    return val;
}

int main(){
    cout<<"Enter expression: ";
    cin>>input;
    int result=E();
    cout<<"Result = "<<result<<endl;
    return 0;
}
```

OUTPUT:

```
Enter expression: 4+4*6-2
Result = 26
```

CONCLUSION: The recursive descent calculator successfully evaluates arithmetic expressions using recursive parsing functions.

Vanya Thakur, A2305223337

EXPERIMENT 13

AIM: Consider the following grammar:

S → ABC

A → abA | ab

B → b | BC

C → c | cC

Following any suitable parsing technique (prefer top-down), design a parser which accepts a string and tells whether the string is accepted by above grammar or not.

SOFTWARE USED: VS Code

THEORY:

A top-down parser starts from the start symbol and attempts to derive the input string using the grammar productions. Recursive descent parsing is a common top-down technique where each non-terminal is implemented as a function.

For the given grammar

S → ABC
A → abA | ab
B → b | BC
C → c | cC

The parser checks the input string step by step using recursive functions corresponding to A, B, and C.

The language generated by this grammar consists of:

- one or more occurrences of ab
- followed by one or more b
- followed by one or more c

ALGORITHM:

```
BEGIN
INPUT string
SET position ← 0

FUNCTION S
  IF A AND B AND C succeed
    RETURN TRUE
  ELSE
    RETURN FALSE
END FUNCTION

FUNCTION A
  IF next symbols are "ab"
    MOVE position by 2
    IF A succeeds
      RETURN TRUE
    ELSE
      RETURN TRUE
  ELSE
    RETURN FALSE
END FUNCTION

FUNCTION B
  IF next symbol is 'b'
```

```
        MOVE position
        RETURN TRUE
    ELSE
        RETURN FALSE
END FUNCTION

FUNCTION C
    IF next symbol is 'c'
        MOVE position
        WHILE next symbol is 'c'
            MOVE position
        RETURN TRUE
    ELSE
        RETURN FALSE
END FUNCTION

CALL S
IF S succeeds AND entire string consumed
    PRINT "String Accepted"
ELSE
    PRINT "String Rejected"
END
```

CODE:

```
#include <iostream>

#include <string>

using namespace std;


string str;

int pos=0;


bool A(){

    if(pos+1<str.length() && str[pos]=='a' && str[pos+1]=='b'){

        pos+=2;

        if(A()) return true;

        return true;

    }

    return false;

}


bool B(){

    if(pos<str.length() && str[pos]=='b'){

        pos++;

        return true;

    }

    return false;

}
```

```
bool C(){
    if(pos<str.length() && str[pos]=='c'){
        pos++;
        while(pos<str.length() && str[pos]=='c')
            pos++;
        return true;
    }
    return false;
}

bool S(){
    int start=pos;
    if(A() && B() && C())
        return true;
    pos=start;
    return false;
}

int main(){
    cout<<"Enter string: ";
    cin>>str;

    if(S() && pos==str.length())
        cout<<"String Accepted\n";
    else
```

```
        cout<<"String Rejected\n";


    return 0;
}
```

OUTPUT:

```
Enter string: abcddd
String Rejected
```

CONCLUSION: The parser based on top-down parsing technique successfully checks whether the input string is accepted by the given grammar.

Vanya Thakur, A2305223337

EXPERIMENT 14

AIM: Design a parser which accepts a mathematical expression (containing integers only). If the expression is valid, then evaluate the expression else report that the expression is invalid.

SOFTWARE USED: VS Code

THEORY:

In compiler design, expression parsing is used to check whether an arithmetic expression follows the grammar rules of a language. A shift reduce parser is a bottom-up parsing technique that uses a stack and performs two main operations: shift and reduce.

Shift means pushing the next input symbol onto the stack. Reduce means replacing a sequence of symbols on the stack that matches the right-hand side of a production rule with the corresponding non-terminal on the left-hand side.

For arithmetic expressions the grammar can be defined as:

$E \rightarrow E + T$
$E \rightarrow E - T$
$E \rightarrow T$
$T \rightarrow T * F$
$T \rightarrow T / F$
$T \rightarrow F$
$F \rightarrow (E)$
$F \rightarrow id$

Here
E represents expression
T represents term
F represents factor

The parser reads the expression, shifts symbols into the stack, reduces them using grammar rules, and finally evaluates the expression.

ALGORITHM:

```
BEGIN
INPUT expression
SET position ← 0

FUNCTION E
   value ← T
   WHILE next symbol is '+' OR '-'
      operator ← next symbol
      MOVE position
      value2 ← T
      IF operator is '+'
         value ← value + value2
      ELSE
         value ← value - value2
   RETURN value
END FUNCTION

FUNCTION T
   value ← F
   WHILE next symbol is '*' OR '/'
      operator ← next symbol
      MOVE position
      value2 ← F
      IF operator is '*'
```

```
        value ← value * value2
    ELSE
        value ← value / value2
  RETURN value
END FUNCTION


FUNCTION F
  IF next symbol is '('
    MOVE position
    value ← E
    IF next symbol is ')'
      MOVE position
      RETURN value
    ELSE
      REPORT invalid expression
  ELSE IF next symbol is digit
    value ← number read from input
    RETURN value
  ELSE
    REPORT invalid expression
END FUNCTION


CALL E
IF entire expression consumed
  PRINT evaluated result
```

ELSE

PRINT "Invalid Expression"

END

CODE:

```
#include <iostream>

#include <string>

#include <cctype>

using namespace std;


string s;

int pos=0;


int E();

int T();

int F();


int E(){

    int val=T();

    while(pos<s.length() && (s[pos]=='+' || s[pos]=='-')){

        char op=s[pos++];

        int val2=T();

        if(op=='+') val+=val2;

        else val-=val2;

    }
```

```
    return val;
}

int T(){
    int val=F();
    while(pos<s.length() && (s[pos]=='*' || s[pos]=='/')){
        char op=s[pos++];
        int val2=F();
        if(op=='*') val*=val2;
        else val/=val2;
    }
    return val;
}

int F(){
    if(pos<s.length() && s[pos]=='('){
        pos++;
        int val=E();
        pos++;
        return val;
    }
    int val=0;
    if(pos<s.length() && isdigit(s[pos])){
        while(pos<s.length() && isdigit(s[pos])){
            val=val*10+(s[pos]-'0');
```

```
            pos++;
        }
        return val;
    }
    throw 1;
}

int main(){
    cout<<"Enter expression: ";
    cin>>s;

    try{
        int result=E();
        if(pos==s.length())
            cout<<"Result = "<<result<<endl;
        else
            cout<<"Invalid Expression\n";
    }
    catch(...){
        cout<<"Invalid Expression\n";
    }

    return 0;
}
```

OUTPUT:

```
Enter expression: 2+3*4
Result = 14
```

CONCLUSION: The parser accepts the mathematical expression as input and returns the calculated result as output if the expression is valid.

Vanya Thakur, A2305223337

EXPERIMENT 15 (OPEN ENDED)

AIM: Designing of various types of parsers.

SOFTWARE USED: VS Code

THEORY:

1. **Recursive Descent Parser (Top-Down parser) :**
   A recursive descent parser is a top-down parsing technique used in compiler design. It begins parsing from the start symbol and attempts to derive the input string by applying grammar rules recursively. Each non-terminal in the grammar is implemented as a separate function. The parser processes the input from left to right and verifies whether the string conforms to the grammar structure.
2. **Predictive Parser (LL(1) Parser):**
   A predictive parser is a type of top-down parser that uses a parsing table to decide which production rule to apply. It reads the input from left to right and constructs a leftmost derivation without backtracking. The parser uses a stack along with the input string and parsing table. Predictive parsing is efficient and commonly used for grammars that satisfy LL(1) conditions.
3. **Shift Reduce Parser (Bottom-Up parser):**
   A shift reduce parser is a bottom-up parsing technique used in syntax analysis. It builds the parse tree from the leaves to the root by shifting input symbols onto a stack and reducing them according to grammar productions. The parser performs two main operations: shift and reduce. Shift pushes input symbols onto the stack, while reduce replaces symbols with non-terminals based on production rules.

ALGORITHM:

BEGIN

INPUT string

SELECT parsing technique

IF technique = Recursive Descent Parser

```
    START from start symbol function
    FOR each non-terminal
        CREATE a recursive function
    MATCH input symbols according to grammar rules
    IF entire input is consumed successfully
        PRINT Accepted
    ELSE
        PRINT Rejected
ELSE IF technique = Predictive Parser
    INITIALIZE stack with start symbol and end marker
    ADD end marker to input
    WHILE stack not empty
        IF top of stack equals current input symbol
            POP stack
            MOVE input pointer
        ELSE
            USE parsing table to replace non-terminal
            PUSH right-hand side of production onto stack
    IF input completely matched
        PRINT Accepted
    ELSE
        PRINT Rejected
ELSE IF technique = Shift Reduce Parser
    INITIALIZE empty stack
    WHILE input not finished
```

```
        SHIFT next input symbol onto stack
        CHECK if stack top matches any production RHS
        IF match found
            REDUCE it to corresponding LHS
    IF stack contains only start symbol
        PRINT Accepted
    ELSE
        PRINT Rejected
END
```

CODE (RECURSIVE DESCENT PARSER):

```
#include <iostream>
#include <string>
using namespace std;

string input;
int pos=0;

bool E();
bool E1();
bool T();
bool T1();
bool F();

bool E(){
```

```
    if(T())
        return E1();
    return false;
}

bool E1(){
    if(pos<input.length() && input[pos]=='+'){
        pos++;
        if(T())
            return E1();
        return false;
    }
    return true;
}

bool T(){
    if(F())
        return T1();
    return false;
}

bool T1(){
    if(pos<input.length() && input[pos]=='*'){
        pos++;
        if(F())
```

```
            return T1();
        return false;
    }
    return true;
}

bool F(){
    if(pos<input.length() && input[pos]=='i'){
        pos++;
        return true;
    }
    if(pos<input.length() && input[pos]=='('){
        pos++;
        if(E() && pos<input.length() && input[pos]==')'){
            pos++;
            return true;
        }
    }
    return false;
}

int main(){

    cout<<"Grammar Used:\n";
    cout<<"E -> T E'\n";
```

```
    cout<<"E' -> + T E' | ε\n";
    cout<<"T -> F T'\n";
    cout<<"T' -> * F T' | ε\n";
    cout<<"F -> (E) | i\n\n";

    cout<<"Enter string: ";
    cin>>input;

    if(E() && pos==input.length())
        cout<<"Accepted\n";
    else
        cout<<"Rejected\n";

    return 0;
}
```

OUTPUT (RECURSIVE DESCENT PARSER):

```
Grammar Used:
E -> T E'
E' -> + T E' | ε
T -> F T'
T' -> * F T' | ε
F -> (E) | i

Enter string: i+i*i
Accepted
```

CODE (PREDICTIVE PARSER):

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;


int main(){


    cout<<"Grammar Used:\n";

    cout<<"E -> T R\n";

    cout<<"R -> + T R | ε\n";

    cout<<"T -> F Y\n";

    cout<<"Y -> * F Y | ε\n";

    cout<<"F -> (E) | i\n\n";


    string input;

    cout<<"Enter string: ";

    cin>>input;


    stack<char> st;

    st.push('$');

    st.push('E');


    input.push_back('$');

    int i=0;
```

```
while(!st.empty()){

    char top=st.top();
    char cur=input[i];

    if(top==cur){
        st.pop();
        i++;
    }

    else if(top=='E'){
        st.pop();
        st.push('R');
        st.push('T');
    }

    else if(top=='R'){
        st.pop();
        if(cur=='+'){
            st.push('R');
            st.push('T');
            st.push('+');
        }
    }
```

```
else if(top=='T'){
    st.pop();
    st.push('Y');
    st.push('F');
}

else if(top=='Y'){
    st.pop();
    if(cur=='*'){
        st.push('Y');
        st.push('F');
        st.push('*');
    }
}

else if(top=='F'){
    st.pop();
    if(cur=='i')
        st.push('i');
    else if(cur=='('){
        st.push(')');
        st.push('E');
        st.push('(');
    }
```

```
            else{
                cout<<"Rejected\n";
                return 0;
            }
        }

        else{
            cout<<"Rejected\n";
            return 0;
        }
    }

    if(input[i]=='$')
        cout<<"Accepted\n";
    else
        cout<<"Rejected\n";

    return 0;
}
```

OUTPUT (PREDICTIVE PARSER):

```
Grammar Used:
E -> T E'
E' -> + T E' | ╬
T -> F T'
T' -> * F T' | ╬
F -> (E) | i

Enter string: (i+i)*i
Accepted
```

CODE (SHIFT REDUCE PARSER):

```
#include <iostream>

#include <string>

using namespace std;


int main(){


    cout<<"Grammar Used:\n";

    cout<<"E -> E + E\n";

    cout<<"E -> E * E\n";

    cout<<"E -> i\n\n";


    string input, stack="";

    int i=0;


    cout<<"Enter string: ";

    cin>>input;
```

```
while(true){

    if(i<input.length()){
        stack+=input[i];
        i++;
    }

    if(stack.find("i")!=string::npos){
        int p=stack.find("i");
        stack.replace(p,1,"E");
    }

    if(stack.find("E+E")!=string::npos){
        int p=stack.find("E+E");
        stack.replace(p,3,"E");
    }

    if(stack.find("E*E")!=string::npos){
        int p=stack.find("E*E");
        stack.replace(p,3,"E");
    }

    if(i>=input.length())
        break;
```

```
    }

    if(stack=="E")
        cout<<"Accepted\n";
    else
        cout<<"Rejected\n";

    return 0;
}
```

OUTPUT (SHIFT REDUCE PARSER):

```
Grammar Used:
E -> E + E
E -> E * E
E -> i

Enter string: i+i*i
Accepted
```

CONCLUSION: The different types of parsers such as recursive descent parser, predictive parser, and shift reduce parser were successfully designed and implemented using C++.